WEEK 1 · DAY 5 · OVERVIEW

Case Study

Bagging and the Spread Cross-Section



Created by **Jeff Lenamon**

ABOUT 50 MIN WITH EXERCISES / 12 SECTIONS / 4 EXERCISES

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

Problem Statement

BUSINESS CONTEXT

Course 4's Day 5 ended with a least squares model of log OAS, Course 4's line, scored in bp on five folds of its 615 training bonds.

Week 1 built bagging and random forests on the card file.

The desk now asks the same of the bonds: does an average of trees, which finds its own bends, describe them better than the line on the same five folds?

It wants the answer for a known issuer and for a new one, and what the forest leans on.

The bonds and issuers come from the course's generator, which Course 4 explained, so every fitted spread describes these rows, never a view on any bond.

OBJECTIVES

Fit a regression tree and a forest on Course 4's folds and score them in bp.

Watch a forest's miss change as trees are averaged, read its out-of-bag score, and set its columns per split and leaf size by out-of-bag error.

Compare the forest with the line without and with coupon, and on folds that keep issuers whole.

Read permutation importance in bp and find the bonds the forest misses most.

ABOUT 50 MIN WITH EXERCISES / 12 SECTIONS / 4 EXERCISES

Your Route Through Day 5

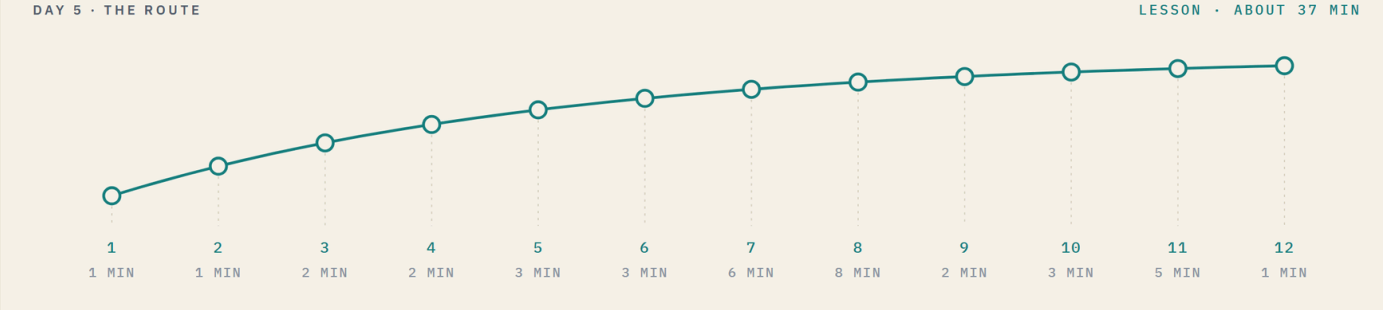


FIGURE 1 Twelve sections, in order, with about how long each takes.

	SECTION	ABOUT
1	Setting Up	1 min
2	Loading the Data	1 min
3	Data Overview	2 min
4	Exploratory Data Analysis	2 min
5	Data Preparation for Modeling	3 min
6	One Regression Tree	3 min

	SECTION	ABOUT
7	Averaging Trees	6 min
8	Model Performance Improvement	8 min
9	Model Performance Comparison	2 min
10	The Split by Issuer	3 min
11	Reading the Tuned Forest	5 min
12	Insights and Conclusion	1 min

Time. About 37 minutes for the lesson, then about 12 for the four exercises.

Fix the seed and load the libraries.

Setting Up

THE SETUP

```
import os
from pathlib import Path
os.environ["OMP_NUM_THREADS"] = "1"
os.environ["OPENBLAS_NUM_THREADS"] = "1"
SEED = 42
DATA = "../../../data/"
if not Path(DATA + "benchmark.csv").exists():
    DATA = "https://raw.githubusercontent.com/lenamonj/python-fixed-income/main/data/"
```

THE LIBRARIES

```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split, KFold, GroupKFold, cross_validate, cross_val_predict
from sklearn.linear_model import LinearRegression
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import BaggingRegressor, RandomForestRegressor
```

Loading the Data

THE DATA DICTIONARY

- **`cusip`**: the bond's identifier
- **`issuer`** and **`ticker`**: the issuer's name and short code
- **`sector`**: one of 10 industry sectors
- **`rating`**: credit rating, AAA to CCC
- **`coupon`**: annual coupon, percent of par
- **`maturity`** and **`as_of_date`**: dates; years to maturity is built from them
- **`oas`**: option-adjusted spread, bp; **the target is its log**
- **`duration`**: modified duration, years
- **`amount_outstanding`**: face value outstanding, dollars

READING THE BONDS

```
bonds = pd.read_csv(DATA + "benchmark.csv")
ratings = pd.read_csv(DATA + "ratings.csv")
```

THE SPLIT

```
train, test = train_test_split(data, test_size=0.25, random_state=SEED)
y = train["log_oas"]
print("Training bonds:", len(train), " Test bonds:", len(test))
```

```
Training bonds: 615  Test bonds: 206
```

The 206 test bonds were Course 4's; this course never scores them. Everything below lives inside the 615.

Data Overview

THE TRAINING BONDS

	OAS	SCORE	YEARS_TO_MATURITY	DURATION	AMOUNT_BN	COUPON
count	615.00	615.00	615.00	615.00	615.00	615.00
mean	180.67	9.21	8.76	5.92	1.39	5.87
std	192.74	3.57	6.79	3.22	0.66	1.70
min	15.00	1.00	0.96	0.92	0.30	1.50

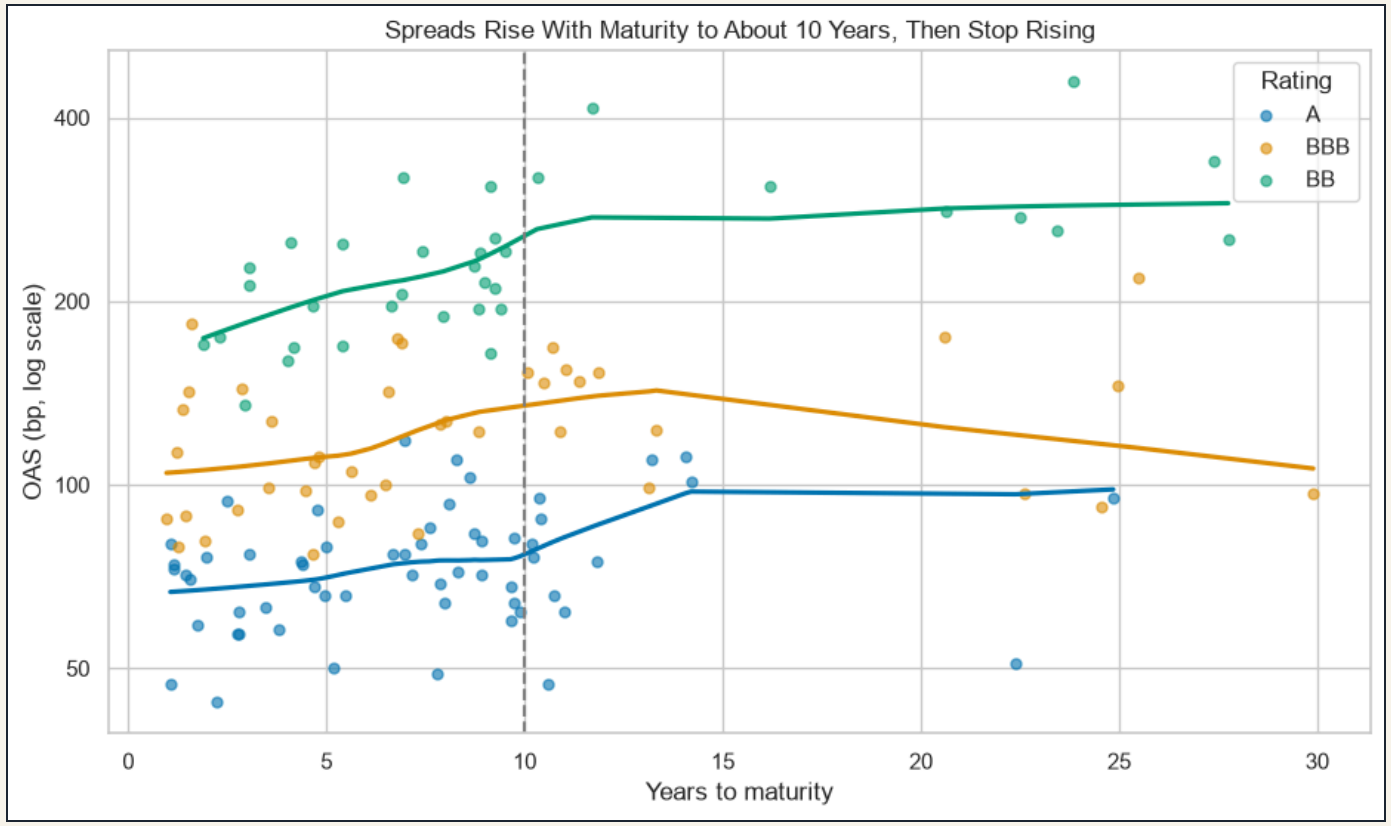
BONDS PER ISSUER

```
print("High yield:", round((train["grade"] == "High Yield").mean() * 100, 1), "percent")
per_issuer = train.groupby("ticker").size()
print("Issuers:", len(per_issuer), " Bonds per issuer:", per_issuer.min(), "to", per_issuer.max(), " Median:", per_issuer.median())
```

```
High yield: 34.3 percent
Issuers: 149  Bonds per issuer: 1 to 8  Median: 4.0
```

Familiar bonds and familiar numbers. What we need is Course 4's line to beat.

Exploratory Data Analysis



THE QUESTION

Q. Does a bond's spread climb with maturity in a straight line?

MEDIAN OAS IN BP BY MATURITY BAND

YEARS_TO_MATURITY	(0, 3]	(3, 6]	(6, 10]
rating			
A	70.00	68.00	77.00
BB	170.00	204.00	215.00
BBB	102.00	102.00	126.00

Spreads rise to about 10 years and then stop. A straight line in years cannot stop; a tree can.

Data Preparation for Modeling

THE TREES' INPUTS

```
X = pd.concat([train[NUMBERS], pd.get_dummies(train["sector"], dtype=float)], axis=1)
print(X.shape)
```

```
(615, 14)
```

THE LINE TO BEAT

```
line = make_model(LinearRegression(), LINE_NUMBERS)
scores = cross_validate(line, LINE_X, y, cv=folds, scoring=bp_scorer, return_train_score=True, n_jobs=1)
held = {"line": -scores["test_score"]}
print("Held-out folds:", held["line"].round(1), " Mean:", round(held["line"].mean(), 2), "bp")
print("Fitted folds, mean:", round(fitted["line"], 2), "bp")
```

```
Held-out folds: [30.1 36.2 61.8 39.3 41.2]  Mean: 41.72 bp
Fitted folds, mean: 39.09 bp
```

Course 4's line misses a held-out bond by 41.7 bp and its own bonds by 39.1. Now one tree.

One Regression Tree

THE QUESTION

Q. Write down both numbers for a full tree.

A FULL TREE ON THE SAME FOLDS

```
full_tree = DecisionTreeRegressor(random_state=SEED)
scores = cross_validate(full_tree, X, y, cv=folds, scoring=bp_scorer, return_train_score=True, n_jobs=1)
```

A SHALLOW TREE

```
shallow_tree = DecisionTreeRegressor(max_depth=4, random_state=SEED)
scores = cross_validate(shallow_tree, X, y, cv=folds, scoring=bp_scorer, return_train_score=True, n_jobs=1)
print("Leaves:", shallow_tree.fit(X, y).get_n_leaves())
print("Held-out:", round(held["depth-4 tree"].mean(), 2), " Fitted:", round(fitted["depth-4 tree"], 2), "bp")
```

```
Leaves: 16
Held-out: 45.21  Fitted: 40.43 bp
```

Averaging Trees

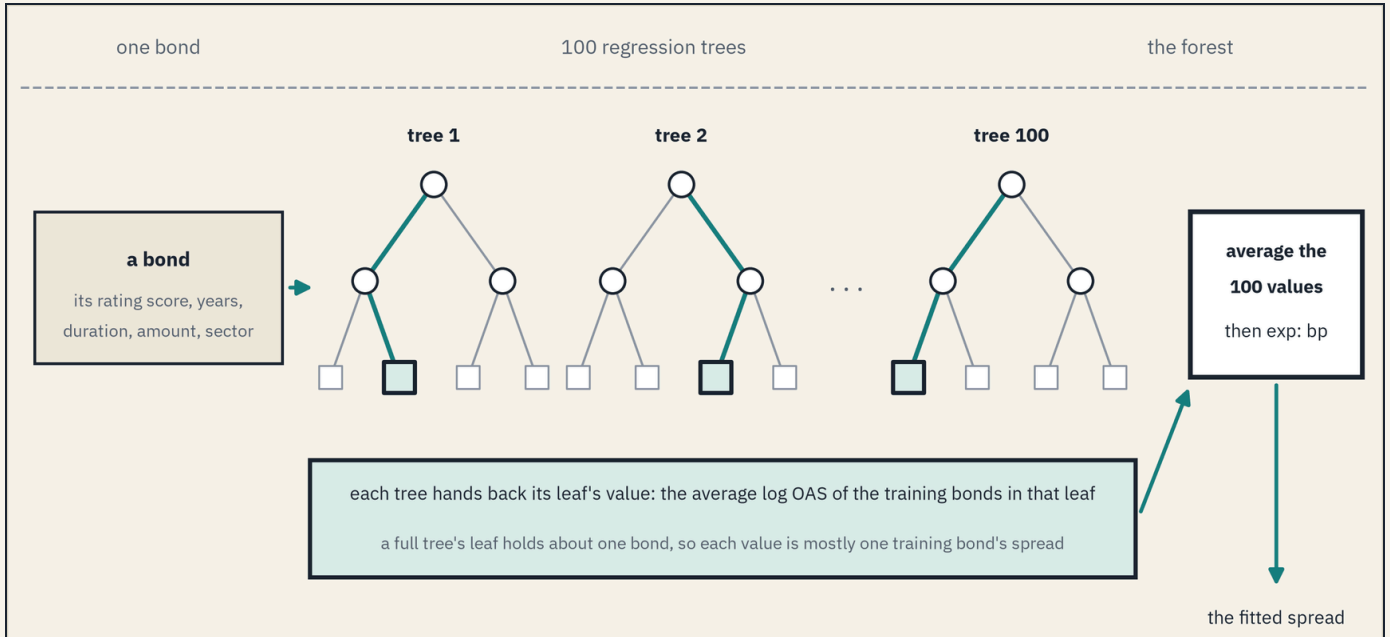


FIGURE 2 One bond goes down 100 trees; each leaf gives a log OAS, and exp of their average is the fitted spread in bp.

THE FIRST QUESTION

Q. How many trees must be averaged before the forest's held-out miss falls below the line's 41.72 bp? Write down a number.

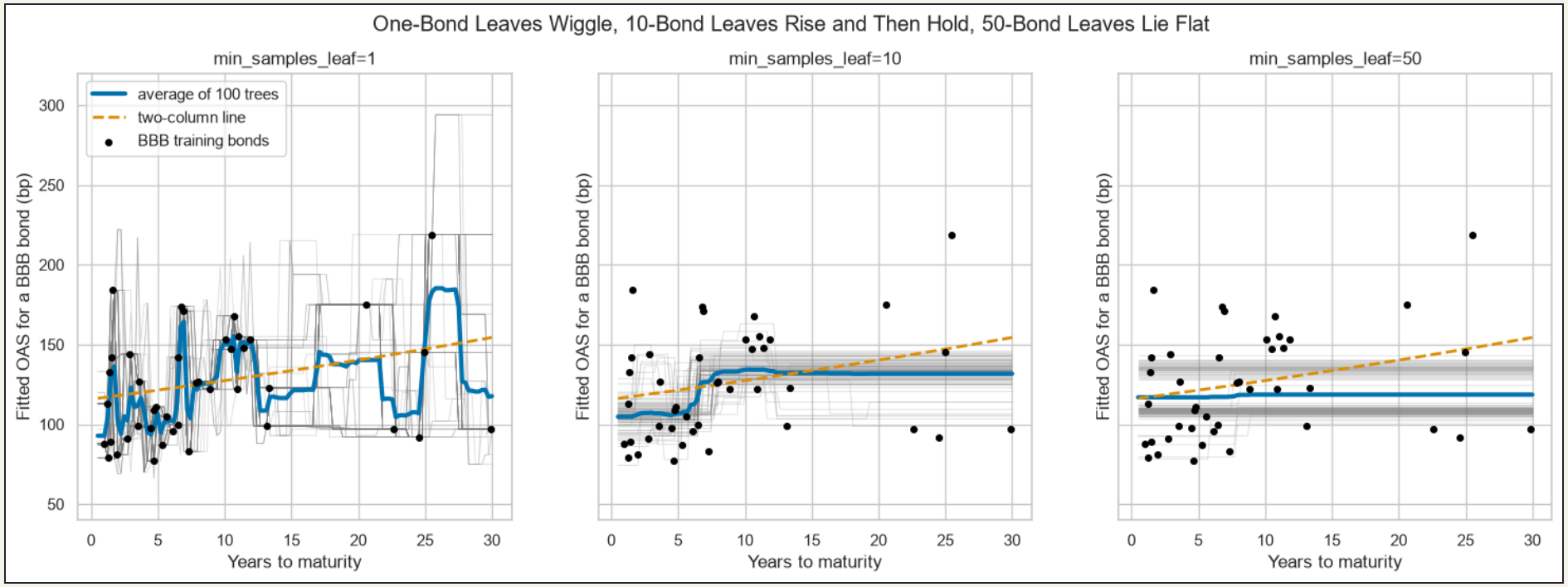
THE SECOND QUESTION

Q. From one fit on all 615 bonds, what is its out-of-bag MAE in bp? Write down a number.

BAGGING, A FOREST, AND THE OUT-OF-BAG OUTPUTS

```
bagging = BaggingRegressor(DecisionTreeRegressor(random_state=SEED), n_estimators=100, random_state=SEED, n_jobs=1)
forest = RandomForestRegressor(random_state=SEED, n_jobs=1)
oob_forest = RandomForestRegressor(oob_score=True, random_state=SEED, n_jobs=1).fit(X, y)
```

Model Performance Improvement



THE QUESTION

Q. Write down the held-out MAE of the same forest with `max_features="sqrt"`.

The line keeps climbing, 116.9 to 154.4, because it cannot stop; with 50-bond leaves the forest hardly moves at all.

Model Performance Comparison

THE QUESTION

Q. Write down the tuned forest's held-out MAE with coupon.

WHAT THE LINE GAINS

With coupon the line's miss falls from 41.72 to 37.19 bp.

COUPON ADDED TO BOTH INPUTS, THEN FOUR MODELS ON THE SAME FOLDS

```
X_coupon = X.assign(coupon=train["coupon"])
LINE_X_COUPON = LINE_X.assign(coupon=train["coupon"])
line_coupon = make_model(LinearRegression(), LINE_NUMBERS + ["coupon"])
held_coupon = {"line": -cross_validate(line_coupon, LINE_X_COUPON, y, cv=folds, scoring=bp_scorer, n_jobs=1)["test_score"]}
for name, model in [("full tree", full_tree), ("forest", forest), ("tuned forest", tuned)]:
    held_coupon[name] = -cross_validate(model, X_coupon, y, cv=folds, scoring=bp_scorer, n_jobs=1)["test_score"]
```

EACH MODEL'S MEAN MISS, WITHOUT AND WITH COUPON

```
comparison = pd.DataFrame({"without coupon": {name: held[name].mean() for name in held_coupon},
                           "with coupon": {name: held_coupon[name].mean() for name in held_coupon}})
comparison
```

Score every model on issuers it never saw.

The Split by Issuer

FOLDS THAT KEEP ISSUERS WHOLE

```
by_issuer = GroupKFold(n_splits=5)
print("Bonds per fold:", [len(rows) for fit_rows, rows in by_issuer.split(X, y, groups=train["ticker"])])
def issuer_mae(model, rows):
    return -cross_validate(model, rows, y, cv=by_issuer, groups=train["ticker"], scoring=bp_scorer, n_jobs=1)["test_score"]
```

```
Bonds per fold: [123, 123, 123, 123, 123]
```

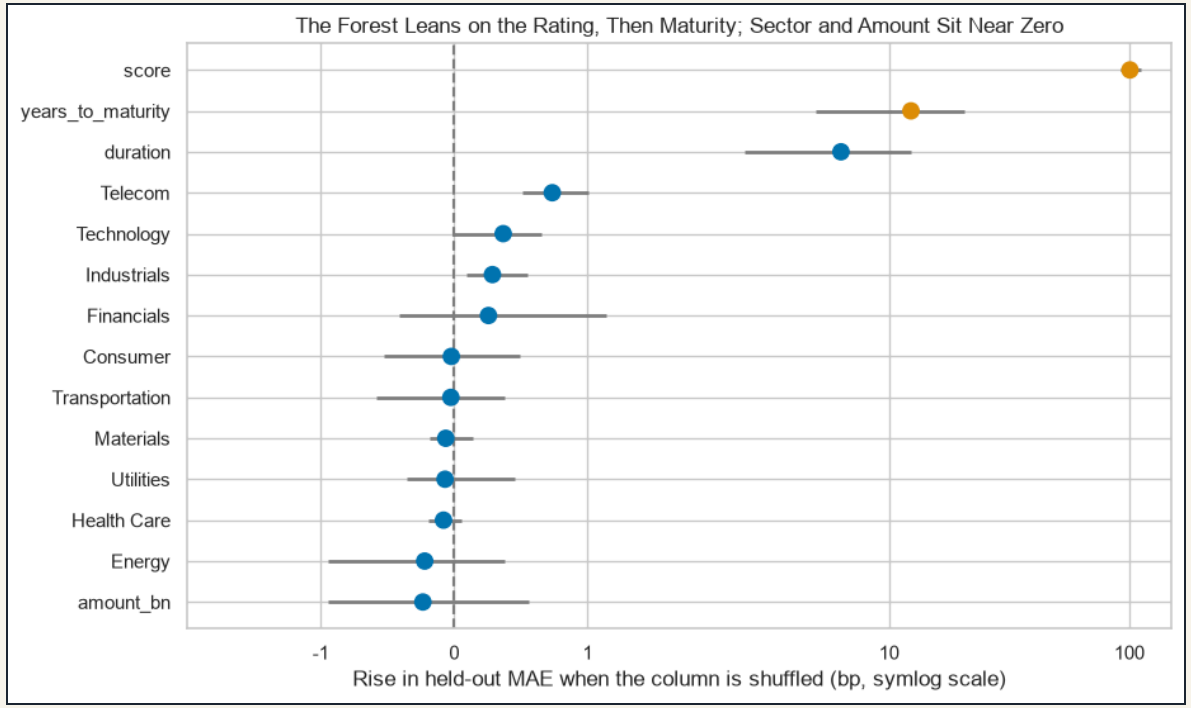
WHAT A HELD-OUT FOLD HOLDS

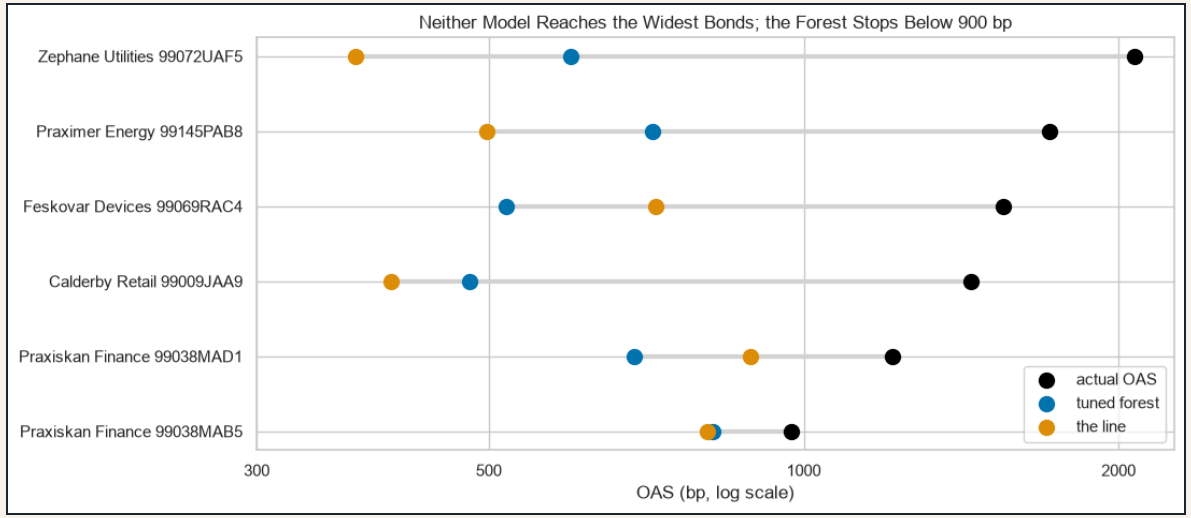
Course 4's Day 5 **`GroupKFold`** keeps each issuer's bonds in one fold, so every held-out fold holds only issuers its model has never seen.

NEW ISSUERS, WITHOUT COUPON

	WITHOUT COUPON
line	45.76
full tree	56.52
forest	45.19
tuned forest	43.37

Reading the Tuned Forest



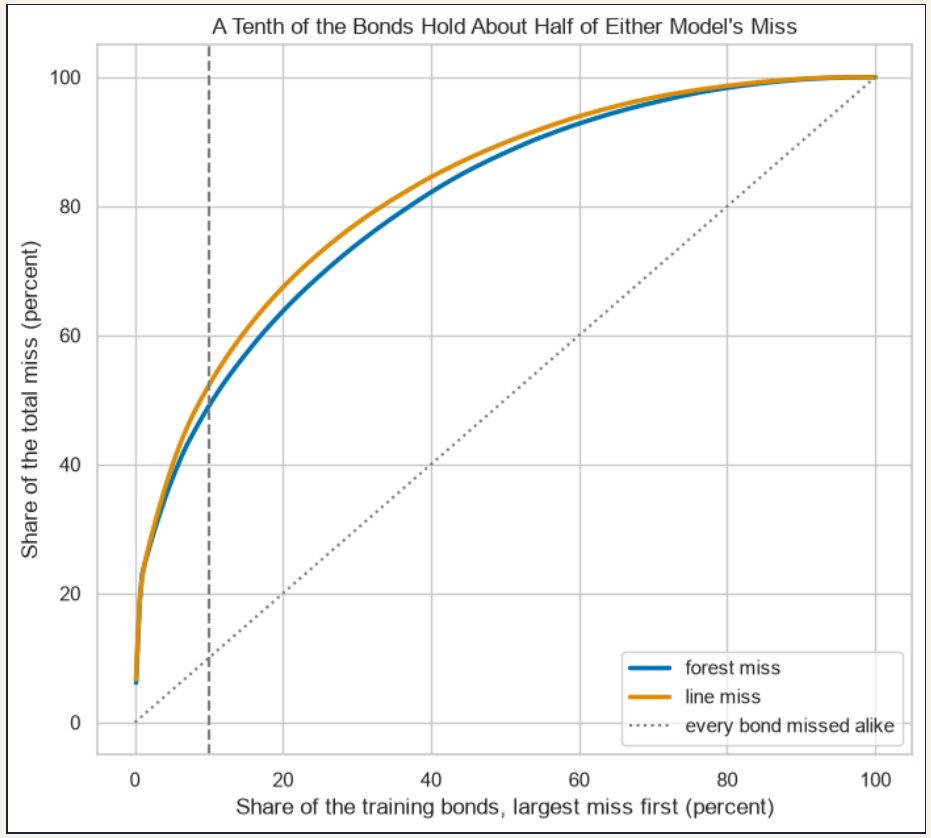


MEAN MISS IN BP BY GRADE

GRADE	FOREST MISS	LINE MISS
High Yield	77.72	85.89
Investment Grade	18.80	18.65

An importance says what this forest leans on, an association in this file, never a driver of spreads.

Insights and Conclusion



THE SHARE OF THE MISS

```
forest miss  largest 5: 20.8  largest 62: 49.2
line miss  largest 5: 20.7  largest 62: 52.4
```

READING THE CURVE

This is a **concentration curve**: the further it bows above the diagonal, the more of the total miss sits in a few bonds.

The five largest alone hold about a fifth of either.

A miss is the model's, never a view on a bond.

Week 1 ends with an average of trees that describes these bonds a little better than a line, and an honest measure of how little. Week 2 grows trees one after another.

You Can Now

- Fit regression trees, bagging and forests and score them in bp on Course 4's folds.
- Read fitted folds against held-out folds with **`return_train_score=True`**.
- Average a forest's trees one at a time with **`.estimators_`**.
- Read a regression forest's **`.oob_prediction_`** and **`.oob_score_`**, and choose settings by out-of-bag error.
- Compare models without and with a column, and on folds that keep issuers whole.
- Read permutation importance in bp and find where a model misses most.

NEXT

WEEK 2 · DAY 1

AdaBoost

Week 2 grows trees one after another, each fixing what the last one missed.